

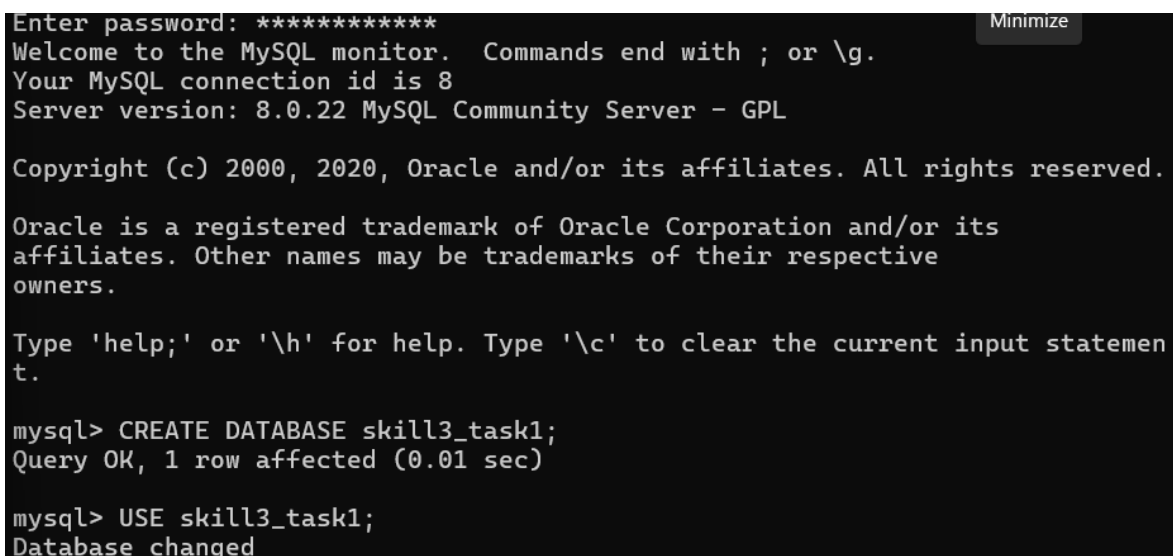
2520090212 J Rakshitha

Skill Session 3

Task 1

DML: SELECT, INSERT, UPDATE, DELETE; parameterised queries to prevent SQL injection ·
JOINS (INNER, LEFT, RIGHT, FULL, CROSS, SELF); subqueries; CTEs (WITH clause)

CREATE DATABASE



```
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.22 MySQL Community Server - GPL

Copyright (c) 2000, 2020, Oracle and/or its affiliates. All rights reserved.

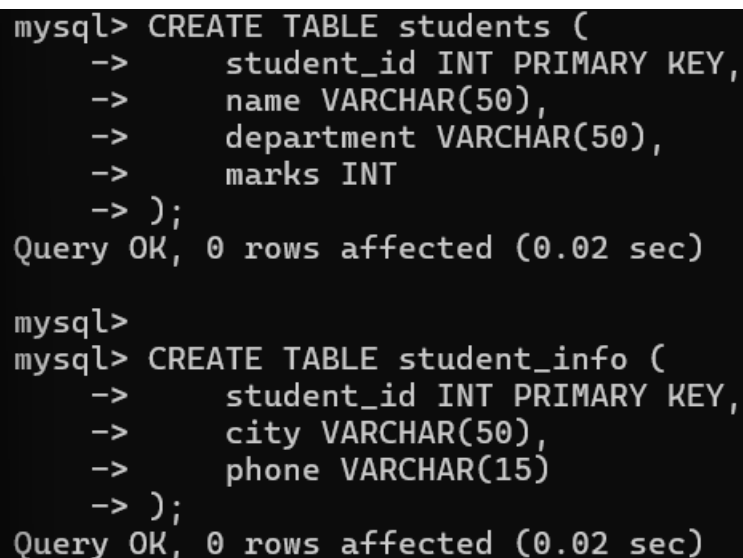
Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE skill3_task1;
Query OK, 1 row affected (0.01 sec)

mysql> USE skill3_task1;
Database changed
```

CREATE TABLES



```
mysql> CREATE TABLE students (
  ->     student_id INT PRIMARY KEY,
  ->     name VARCHAR(50),
  ->     department VARCHAR(50),
  ->     marks INT
  -> );
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> CREATE TABLE student_info (
  ->     student_id INT PRIMARY KEY,
  ->     city VARCHAR(50),
  ->     phone VARCHAR(15)
  -> );
Query OK, 0 rows affected (0.02 sec)
```

INSERT

```
mysql> INSERT INTO students VALUES
-> (1, 'Abhi', 'CSE', 85),
-> (2, 'Adam', 'ECE', 92),
-> (3, 'Alex', 'CSE', 78),
-> (4, 'Ashish', 'IT', 95),
-> (5, 'Chester', 'MECH', 68);
Query OK, 5 rows affected (0.02 sec)
Records: 5 Duplicates: 0 Warnings: 0
```

```
mysql>
mysql> INSERT INTO student_info VALUES
-> (1, 'Delhi', '9876543210'),
-> (2, 'Mumbai', '9876543211'),
-> (3, 'Chennai', '9876543212'),
-> (6, 'Bangalore', '9876543213');
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT * FROM students;
+-----+-----+-----+-----+
| student_id | name   | department | marks |
+-----+-----+-----+-----+
| 1          | Abhi   | CSE        | 85    |
| 2          | Adam   | ECE        | 92    |
| 3          | Alex   | CSE        | 78    |
| 4          | Ashish | IT         | 95    |
| 5          | Chester| MECH       | 68    |
+-----+-----+-----+-----+
5 rows in set (0.01 sec)
```

```
mysql>
mysql> SELECT * FROM student_info;
+-----+-----+-----+
| student_id | city      | phone      |
+-----+-----+-----+
| 1          | Delhi    | 9876543210 |
| 2          | Mumbai   | 9876543211 |
| 3          | Chennai  | 9876543212 |
| 6          | Bangalore| 9876543213 |
+-----+-----+-----+
4 rows in set (0.00 sec)
```

SELECT

```
mysql> SELECT * FROM students;
```

student_id	name	department	marks
1	Abhi	CSE	85
2	Adam	ECE	92
3	Alex	CSE	78
4	Ashish	IT	95
5	Chester	MECH	68

```
5 rows in set (0.00 sec)
```



```
mysql> SELECT name, department, marks
-> FROM students;
```

name	department	marks
Abhi	CSE	85
Adam	ECE	92
Alex	CSE	78
Ashish	IT	95
Chester	MECH	68

```
5 rows in set (0.00 sec)
```



```
mysql> SELECT *
-> FROM students
-> WHERE marks > 80;
```

student_id	name	department	marks
1	Abhi	CSE	85
2	Adam	ECE	92
4	Ashish	IT	95

```
3 rows in set (0.00 sec)
```

UPDATE

```
mysql> UPDATE students
-> SET marks = 82
-> WHERE student_id = 3;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1 Changed: 1 Warnings: 0
```



```
mysql> SELECT *
-> FROM students
-> WHERE student_id = 3;
```

student_id	name	department	marks
3	Alex	CSE	82

```
1 row in set (0.00 sec)
```

DELETE

```
mysql> DELETE FROM students
      -> WHERE student_id = 5;
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM students;
+-----+-----+-----+-----+
| student_id | name   | department | marks |
+-----+-----+-----+-----+
|          1 | Abhi   | CSE        | 85    |
|          2 | Adam   | ECE        | 92    |
|          3 | Alex   | CSE        | 82    |
|          4 | Ashish | IT         | 95    |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> INSERT INTO students VALUES
      -> (5, 'Chester', 'MECH', 68);
Query OK, 1 row affected (0.00 sec)
```

PARAMETERISED QUERY

```
mysql> PREPARE stmt
      -> FROM 'SELECT * FROM students WHERE name = ?';
Query OK, 0 rows affected (0.00 sec)
Statement prepared

mysql> SET @student_name = 'Adam';
Query OK, 0 rows affected (0.00 sec)

mysql> EXECUTE stmt USING @student_name;
+-----+-----+-----+-----+
| student_id | name   | department | marks |
+-----+-----+-----+-----+
|          2 | Adam   | ECE        | 92    |
+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> DEALLOCATE PREPARE stmt;
Query OK, 0 rows affected (0.00 sec)
```

SQL INJECTION PREVENTION

```
mysql> SELECT *
-> FROM students
-> WHERE name = '' OR '1'='1';
```

student_id	name	department	marks
1	Abhi	CSE	85
2	Adam	ECE	92
3	Alex	CSE	82
4	Ashish	IT	95
5	Chester	MECH	68

```
5 rows in set (0.00 sec)
```

```
mysql> PREPARE stmt
-> FROM 'SELECT * FROM students WHERE name = ?';
Query OK, 0 rows affected (0.00 sec)
Statement prepared
```

```
mysql>
mysql> SET @student_name = 'Adam';
Query OK, 0 rows affected (0.00 sec)
```

```
mysql>
mysql> EXECUTE stmt USING @student_name;
```

student_id	name	department	marks
2	Adam	ECE	92

```
1 row in set (0.00 sec)
```

```
mysql>
mysql> DEALLOCATE PREPARE stmt;
Query OK, 0 rows affected (0.00 sec)
```

INNER JOIN

```
mysql> SELECT
->     s.student_id,
->     s.name,
->     s.department,
->     i.city
-> FROM students s
-> INNER JOIN student_info i
-> ON s.student_id = i.student_id;
```

student_id	name	department	city
1	Abhi	CSE	Delhi
2	Adam	ECE	Mumbai
3	Alex	CSE	Chennai

3 rows in set (0.00 sec)

LEFT JOIN

```
mysql> SELECT
->     s.student_id,
->     s.name,
->     s.department,
->     i.city
-> FROM students s
-> LEFT JOIN student_info i
-> ON s.student_id = i.student_id;
```

student_id	name	department	city
1	Abhi	CSE	Delhi
2	Adam	ECE	Mumbai
3	Alex	CSE	Chennai
4	Ashish	IT	NULL
5	Chester	MECH	NULL

5 rows in set (0.00 sec)

RIGHT JOIN

```
mysql> SELECT
->     s.student_id,
->     s.name,
->     s.department,
->     i.city
-> FROM students s
-> RIGHT JOIN student_info i
-> ON s.student_id = i.student_id;
```

student_id	name	department	city
1	Abhi	CSE	Delhi
2	Adam	ECE	Mumbai
3	Alex	CSE	Chennai
NULL	NULL	NULL	Bangalore

```
4 rows in set (0.00 sec)
```

FULL JOIN

```

mysql> SELECT
->     s.student_id,
->     s.name,
->     s.department,
->     i.city
-> FROM students s
-> LEFT JOIN student_info i
-> ON s.student_id = i.student_id
->
-> UNION
->
-> SELECT
->     s.student_id,
->     s.name,
->     s.department,
->     i.city
-> FROM students s
-> RIGHT JOIN student_info i
-> ON s.student_id = i.student_id;
+-----+-----+-----+-----+
| student_id | name   | department | city   |
+-----+-----+-----+-----+
|          1 | Abhi   | CSE        | Delhi  |
|          2 | Adam   | ECE        | Mumbai |
|          3 | Alex   | CSE        | Chennai |
|          4 | Ashish | IT         | NULL   |
|          5 | Chester | MECH       | NULL   |
|         NULL | NULL   | NULL       | Bangalore |
+-----+-----+-----+-----+
6 rows in set (0.00 sec)

```

CROSS JOIN

```
mysql> SELECT
->     s.name,
->     i.city
-> FROM students s
-> CROSS JOIN student_info i;
```

name	city
Abhi	Delhi
Abhi	Mumbai
Abhi	Chennai
Abhi	Bangalore
Adam	Delhi
Adam	Mumbai
Adam	Chennai
Adam	Bangalore
Alex	Delhi
Alex	Mumbai
Alex	Chennai
Alex	Bangalore
Ashish	Delhi
Ashish	Mumbai
Ashish	Chennai
Ashish	Bangalore
Chester	Delhi
Chester	Mumbai
Chester	Chennai
Chester	Bangalore

```
20 rows in set (0.00 sec)
```

SELF JOIN

```
mysql> CREATE TABLE employee (  
->     employee_id INT PRIMARY KEY,  
->     employee_name VARCHAR(50),  
->     manager_id INT  
-> );
```

Query OK, 0 rows affected (0.02 sec)

```
mysql> INSERT INTO employee VALUES  
-> (1, 'John', NULL),  
-> (2, 'Alice', 1),  
-> (3, 'Bob', 1),  
-> (4, 'Charlie', 2),  
-> (5, 'David', 2);
```

Query OK, 5 rows affected (0.01 sec)

Records: 5 Duplicates: 0 Warnings: 0

```
mysql> SELECT  
->     e.employee_name AS Employee,  
->     m.employee_name AS Manager  
-> FROM employee e  
-> LEFT JOIN employee m  
-> ON e.manager_id = m.employee_id;
```

Employee	Manager
John	NULL
Alice	John
Bob	John
Charlie	Alice
David	Alice

5 rows in set (0.00 sec)

SUBQUERY

```
mysql> SELECT
->     name,
->     department,
->     marks
-> FROM students
-> WHERE marks > (
->     SELECT AVG(marks)
->     FROM students
-> );
```

name	department	marks
Abhi	CSE	85
Adam	ECE	92
Ashish	IT	95

3 rows in set (0.01 sec)

```
mysql> SELECT AVG(marks)
-> FROM students;
```

AVG(marks)
84.4000

1 row in set (0.00 sec)

ANOTHER SUBQUERY — MAX MARKS

```
mysql> SELECT
->     name,
->     department,
->     marks
-> FROM students
-> WHERE marks = (
->     SELECT MAX(marks)
->     FROM students
-> );
```

name	department	marks
Ashish	IT	95

1 row in set (0.00 sec)

CTE — WITH CLAUSE

```
mysql> WITH high_scorers AS (  
->     SELECT  
->         student_id,  
->         name,  
->         department,  
->         marks  
->     FROM students  
->     WHERE marks > 80  
-> )  
-> SELECT *  
-> FROM high_scorers;  
+-----+-----+-----+-----+  
| student_id | name   | department | marks |  
+-----+-----+-----+-----+  
|          1 | Abhi   | CSE        | 85    |  
|          2 | Adam   | ECE        | 92    |  
|          3 | Alex   | CSE        | 82    |  
|          4 | Ashish | IT         | 95    |  
+-----+-----+-----+-----+  
4 rows in set (0.00 sec)
```

CTE + GROUP BY

```
mysql> WITH department_average AS (  
->     SELECT  
->         department,  
->         AVG(marks) AS average_marks  
->     FROM students  
->     GROUP BY department  
-> )  
-> SELECT *  
-> FROM department_average  
-> WHERE average_marks > 80;  
+-----+-----+  
| department | average_marks |  
+-----+-----+  
| CSE        | 83.5000      |  
| ECE        | 92.0000      |  
| IT         | 95.0000      |  
+-----+-----+  
3 rows in set (0.00 sec)  
  
mysql>
```

Task 2

Aggregate functions; GROUP BY / HAVING; window functions: ROW_NUMBER, RANK, LAG, LEAD · CASE expressions; JSONB operators (->>); UNION / INTERSECT / EXCEPT · PBL: Write 12+ queries matching all frontend dashboard and listing requirements

AGGREGATE FUNCTIONS

- Count

```
mysql> SELECT COUNT(*) AS total_students
-> FROM students;
+-----+
| total_students |
+-----+
|          5 |
+-----+
1 row in set (0.00 sec)
```

- Sum

```
mysql> SELECT SUM(marks) AS total_marks
-> FROM students;
+-----+
| total_marks |
+-----+
|        422 |
+-----+
1 row in set (0.00 sec)
```

- Avg

```
mysql> SELECT AVG(marks) AS average_marks
-> FROM students;
+-----+
| average_marks |
+-----+
|      84.4000 |
+-----+
1 row in set (0.00 sec)
```

- Max

```
mysql> SELECT MAX(marks) AS highest_marks
-> FROM students;
+-----+
| highest_marks |
+-----+
|           95 |
+-----+
1 row in set (0.00 sec)
```

- Min

```
mysql> SELECT MIN(marks) AS lowest_marks
-> FROM students;
+-----+
| lowest_marks |
+-----+
|           68 |
+-----+
1 row in set (0.00 sec)
```

GROUP BY

```
mysql> SELECT
->     department,
->     COUNT(*) AS student_count,
->     AVG(marks) AS average_marks
-> FROM students
-> GROUP BY department;
+-----+-----+-----+
| department | student_count | average_marks |
+-----+-----+-----+
| CSE       | 2             | 83.5000      |
| ECE       | 1             | 92.0000      |
| IT        | 1             | 95.0000      |
| MECH      | 1             | 68.0000      |
+-----+-----+-----+
4 rows in set (0.00 sec)
```

HAVING

```
mysql> SELECT
->     department,
->     AVG(marks) AS average_marks
-> FROM students
-> GROUP BY department
-> HAVING AVG(marks) > 80;
```

department	average_marks
CSE	83.5000
ECE	92.0000
IT	95.0000

3 rows in set (0.00 sec)

WINDOW FUNCTIONS

```
mysql> SELECT
->     name,
->     department,
->     marks,
->     ROW_NUMBER() OVER (
->         ORDER BY marks DESC
->     ) AS rn
-> FROM students;
```

name	department	marks	rn
Ashish	IT	95	1
Adam	ECE	92	2
Abhi	CSE	85	3
Alex	CSE	82	4
Chester	MECH	68	5

5 rows in set (0.04 sec)

RANK()

```
mysql> SELECT
->     name,
->     department,
->     marks,
->     RANK() OVER (
->         ORDER BY marks DESC
->     ) AS rank_no
-> FROM students;
```

name	department	marks	rank_no
Ashish	IT	95	1
Adam	ECE	92	2
Abhi	CSE	85	3
Alex	CSE	82	4
Chester	MECH	68	5

5 rows in set (0.00 sec)

LAG()

```
mysql> SELECT
->     name,
->     department,
->     marks,
->     LAG(marks) OVER (
->         ORDER BY marks DESC
->     ) AS previous_marks
-> FROM students;
```

name	department	marks	previous_marks
Ashish	IT	95	NULL
Adam	ECE	92	95
Abhi	CSE	85	92
Alex	CSE	82	85
Chester	MECH	68	82

5 rows in set (0.00 sec)

LEAD()

```
mysql> SELECT
->     name,
->     department,
->     marks,
->     LEAD(marks) OVER (
->         ORDER BY marks DESC
->     ) AS next_marks
-> FROM students;
```

name	department	marks	next_marks
Ashish	IT	95	92
Adam	ECE	92	85
Abhi	CSE	85	82
Alex	CSE	82	68
Chester	MECH	68	NULL

5 rows in set (0.00 sec)

CASE EXPRESSIONS

```
mysql> SELECT
->     name,
->     marks,
->     CASE
->         WHEN marks >= 90 THEN 'Excellent'
->         WHEN marks >= 75 THEN 'Good'
->         WHEN marks >= 60 THEN 'Average'
->         ELSE 'Needs Improvement'
->     END AS performance
-> FROM students;
```

name	marks	performance
Abhi	85	Good
Adam	92	Excellent
Alex	82	Good
Ashish	95	Excellent
Chester	68	Average

5 rows in set (0.00 sec)

UNION

```
mysql> SELECT name, department
-> FROM students
-> WHERE department = 'CSE'
->
-> UNION
->
-> SELECT name, department
-> FROM students
-> WHERE department = 'ECE';
```

name	department
Abhi	CSE
Alex	CSE
Adam	ECE

3 rows in set (0.00 sec)

INTERSECT

```
mysql> SELECT DISTINCT s.student_id
-> FROM students s
-> INNER JOIN students c
-> ON s.student_id = c.student_id
-> WHERE s.marks >= 80
-> AND c.department = 'CSE';
```

student_id
1
3

2 rows in set (0.00 sec)

EXCEPT

```
mysql> SELECT s.student_id
-> FROM students s
-> WHERE s.marks >= 80
-> AND NOT EXISTS (
->     SELECT 1
->     FROM students c
->     WHERE c.student_id = s.student_id
->     AND c.department = 'CSE'
-> );
```

```
+-----+
| student_id |
+-----+
|          2 |
|          4 |
+-----+
```

2 rows in set (0.00 sec)

QUERY

- Query 1 — Display all students

```
mysql> SELECT *
-> FROM students;
```

```
+-----+-----+-----+-----+
| student_id | name   | department | marks |
+-----+-----+-----+-----+
|          1 | Abhi   | CSE        | 85    |
|          2 | Adam   | ECE        | 92    |
|          3 | Alex   | CSE        | 82    |
|          4 | Ashish | IT         | 95    |
|          5 | Chester| MECH       | 68    |
+-----+-----+-----+-----+
```

5 rows in set (0.00 sec)

- Query 2 — Students scoring above 80

```
mysql> SELECT name, department, marks
-> FROM students
-> WHERE marks > 80;
```

name	department	marks
Abhi	CSE	85
Adam	ECE	92
Alex	CSE	82
Ashish	IT	95

```
4 rows in set (0.00 sec)
```

- Query 3 — Students from CSE

```
mysql> SELECT name, marks
-> FROM students
-> WHERE department = 'CSE';
```

name	marks
Abhi	85
Alex	82

```
2 rows in set (0.00 sec)
```

- Query 4 — Students from IT

```
mysql> SELECT name, marks
-> FROM students
-> WHERE department = 'IT';
```

name	marks
Ashish	95

```
1 row in set (0.00 sec)
```

- Query 5 — Highest scorer

```
mysql> SELECT name, department, marks
-> FROM students
-> WHERE marks = (
->     SELECT MAX(marks)
->     FROM students
-> );
```

name	department	marks
Ashish	IT	95

```
1 row in set (0.00 sec)
```

- Query 6 — Lowest scorer

```
mysql> SELECT name, department, marks
-> FROM students
-> WHERE marks = (
->     SELECT MIN(marks)
->     FROM students
-> );
```

name	department	marks
Chester	MECH	68

```
1 row in set (0.00 sec)
```

- Query 7 — Average marks

```
mysql> SELECT AVG(marks) AS average_marks
-> FROM students;
```

average_marks
84.4000

```
1 row in set (0.00 sec)
```

- Query 8 — Total number of students

```
mysql> SELECT COUNT(*) AS total_students
-> FROM students;
+-----+
| total_students |
+-----+
|          5 |
+-----+
1 row in set (0.00 sec)
```

- Query 9 — Department-wise student count

```
mysql> SELECT
->     department,
->     COUNT(*) AS student_count
-> FROM students
-> GROUP BY department;
+-----+-----+
| department | student_count |
+-----+-----+
| CSE       |          2 |
| ECE       |          1 |
| IT        |          1 |
| MECH      |          1 |
+-----+-----+
4 rows in set (0.00 sec)
```

-
- Query 10 — Department-wise average

```
mysql> SELECT
->     department,
->     AVG(marks) AS average_marks
-> FROM students
-> GROUP BY department;
+-----+-----+
| department | average_marks |
+-----+-----+
| CSE       |      83.5000 |
| ECE       |      92.0000 |
| IT        |      95.0000 |
| MECH      |      68.0000 |
+-----+-----+
4 rows in set (0.00 sec)
```

- Query 11 — Departments with average above 80

```
mysql> SELECT
->     department,
->     AVG(marks) AS average_marks
-> FROM students
-> GROUP BY department
-> HAVING AVG(marks) > 80;
```

department	average_marks
CSE	83.5000
ECE	92.0000
IT	95.0000

3 rows in set (0.00 sec)

- Query 12 — Top 3 students

```
mysql> SELECT
->     name,
->     department,
->     marks
-> FROM students
-> ORDER BY marks DESC
-> LIMIT 3;
```

name	department	marks
Ashish	IT	95
Adam	ECE	92
Abhi	CSE	85

3 rows in set (0.00 sec)

- Query 13 — Student performance category

```
mysql> SELECT
->     name,
->     marks,
->     CASE
->         WHEN marks >= 90 THEN 'Excellent'
->         WHEN marks >= 75 THEN 'Good'
->         WHEN marks >= 60 THEN 'Average'
->         ELSE 'Needs Improvement'
->     END AS performance
-> FROM students;
```

name	marks	performance
Abhi	85	Good
Adam	92	Excellent
Alex	82	Good
Ashish	95	Excellent
Chester	68	Average

5 rows in set (0.00 sec)

- Query 14 — Student ranking

```
mysql> SELECT
->     name,
->     department,
->     marks,
->     RANK() OVER (
->         ORDER BY marks DESC
->     ) AS rank_no
-> FROM students;
```

name	department	marks	rank_no
Ashish	IT	95	1
Adam	ECE	92	2
Abhi	CSE	85	3
Alex	CSE	82	4
Chester	MECH	68	5

5 rows in set (0.00 sec)

- Query 15 — Dashboard summary

```
mysql> SELECT
->     COUNT(*) AS total_students,
->     AVG(marks) AS average_marks,
->     MAX(marks) AS highest_marks,
->     MIN(marks) AS lowest_marks
-> FROM students;
```

total_students	average_marks	highest_marks	lowest_marks
5	84.4000	95	68

1 row in set (0.00 sec)

